

Guía Práctica de SSH — Administración Remota para DevOps

Preparado para Matías González Ferreyra

Agosto 2026

Contents

Introducción	2
Herramientas que vas a necesitar	2
Etapa 1: Conceptos y primera conexión	2
Teoría	2
Ejemplos	2
Tarea / Proyecto: “Primera conexión remota documentada”	3
Etapa 2: Autenticación por llaves (SSH keys)	3
Teoría	3
Ejemplos	3
Tarea / Proyecto: “Acceso sin contraseña”	4
Etapa 3: Configuración del servidor SSH (hardening básico)	4
Teoría	4
Ejemplos	4
Tarea / Proyecto: “Servidor SSH endurecido”	5
Etapa 4: Túneles SSH y reenvío de puertos	5
Teoría	5
Ejemplos	5
Tarea / Proyecto: “Acceso seguro a un servicio remoto”	5
Etapa 5: Transferencia de archivos (SCP y SFTP)	6
Teoría	6
Ejemplos	6
Tarea / Proyecto: “Sincronización de un proyecto”	6
Etapa 6: Configuración del cliente (~/.ssh/config)	7
Teoría	7
Ejemplos	7
Tarea / Proyecto: “Cliente SSH organizado”	7
Proyecto final integrador: “Servidor de práctica asegurado y documentado”	7
Recursos recomendados	8
Cómo subir cada proyecto a GitHub	8

Introducción

SSH (Secure Shell) es el protocolo que permite conectarte y administrar un servidor de forma remota y segura, desde cualquier lugar. Es una habilidad no negociable para DevOps: prácticamente todo servidor en la nube se administra por SSH.

Cada etapa tiene:

1. **Teoría** en palabras simples.
2. **Ejemplos** de comandos.
3. **Tarea o mini-proyecto** documentado en tu repositorio de GitHub.

Herramientas que vas a necesitar

Herramienta	Para qué sirve	Notas
Cliente OpenSSH	Conectarte a servidores remotos	Ya viene instalado en Debian
Servidor OpenSSH (openssh-server)	Permitir que otros se conecten a tu máquina	<code>sudo apt install openssh-server</code>
Una segunda máquina o VM	Para practicar conexiones reales cliente-servidor	Puede ser otra VM en VirtualBox, o una instancia gratuita de la capa gratuita de AWS/Oracle Cloud
fail2ban	Bloquea IPs que intentan acceder por fuerza bruta	<code>sudo apt install fail2ban</code>

Recomendación: practica con dos máquinas virtuales (una como “cliente” y otra como “servidor”), o usa una instancia gratuita en la nube (Oracle Cloud tiene capa gratuita permanente) para tener un servidor real al que conectarte desde tu Debian.

Etapa 1: Conceptos y primera conexión

Teoría

SSH funciona bajo un modelo **cliente-servidor**: tu computadora (cliente) se conecta a otra máquina (servidor) que tiene el servicio SSH activo, escuchando por el puerto 22 por defecto.

- `ssh usuario@direccion_ip` — conecta al servidor con ese usuario.
- La primera vez que te conectas a un servidor nuevo, SSH te muestra una “huella digital” (fingerprint) y te pregunta si confías en ella — esto protege contra ataques donde alguien se hace pasar por el servidor real.
- Puedes salir de una sesión SSH con el comando `exit`.

Piensa en SSH como llamar por teléfono a un servidor: marcas su dirección (IP), te identificas (usuario/contraseña o llave), y una vez que te contesta, puedes darle instrucciones como si estuvieras sentado frente a él.

Ejemplos

```

# Instalar el servidor SSH en la máquina que hará de "servidor"
sudo apt install openssh-server
sudo systemctl status ssh          # verificar que está corriendo

# Desde el cliente, conectarte al servidor (reemplaza con la IP real)
ssh matias@192.168.1.50

# Primera vez: SSH pregunta si confías en la huella digital
# Escribe "yes" y presiona Enter

# Una vez dentro, ya estás usando la terminal del servidor remoto
whoami
exit                                # salir de la sesión remota

```

Tarea / Proyecto: “Primera conexión remota documentada”

Configura dos máquinas (o una VM y tu equipo principal). En la que hará de servidor, instala openssh-server. Desde la otra, conéctate por SSH. Documenta en conexion_etapa1.md:

1. Cómo obtuviste la IP del servidor (ip a).
2. El comando exacto que usaste para conectarte.
3. Qué viste la primera vez (la pregunta de la huella digital) y por qué existe esa validación.

Sube el documento a un nuevo repositorio en GitHub llamado ssh-devops-practica.

Etapa 2: Autenticación por llaves (SSH keys)

Teoría

Conectarte con usuario y contraseña funciona, pero es menos seguro (las contraseñas se pueden adivinar o robar). La forma profesional es usar un **par de llaves criptográficas**:

- **Llave privada** — se queda en tu computadora, nunca se comparte. Es como tu identificación personal.
- **Llave pública** — se copia al servidor al que quieres conectarte. Es como darle al servidor una copia de tu huella digital para que te reconozca.
- ssh-keygen — genera el par de llaves.
- ssh-copy-id — copia tu llave pública al servidor automáticamente.

Una vez configuradas las llaves, puedes conectarte sin escribir contraseña, y es mucho más seguro porque nadie puede “adivinar” una llave criptográfica como sí puede adivinar una contraseña débil.

Ejemplos

```

# Generar un par de llaves (en tu máquina cliente)
ssh-keygen -t ed25519 -C "matias@devops-practica"
# Presiona Enter para aceptar la ubicación por defecto (~/.ssh/id_ed25519)
# Puedes poner una frase de contraseña (passphrase) opcional para más seguridad

# Copiar tu llave pública al servidor
ssh-copy-id matias@192.168.1.50

```

```
# Ahora puedes conectarte sin contraseña
ssh matias@192.168.1.50

# Ver tus llaves generadas
ls -la ~/.ssh/
cat ~/.ssh/id_ed25519.pub           # esta es la que se comparte (pública)
```

Tarea / Proyecto: “Acceso sin contraseña”

1. Genera tu propio par de llaves con ssh-keygen.
 2. Cópialas a tu servidor de práctica con ssh-copy-id.
 3. Confirma que puedes conectarte sin que te pida contraseña.
 4. Documenta en conexion_etapa2.md la diferencia entre llave pública y privada, con tus propias palabras, como si se lo explicarás a alguien sin conocimientos técnicos.
-

Etapa 3: Configuración del servidor SSH (hardening básico)

Teoría

El archivo de configuración del servidor SSH vive en /etc/ssh/sshd_config. Ajustarlo correctamente es una de las tareas más importantes de seguridad en DevOps:

- PermitRootLogin no — evita que alguien se conecte directamente como usuario root (administrador), obligando a usar un usuario normal y luego sudo.
- PasswordAuthentication no — desactiva el login por contraseña, forzando el uso de llaves (solo hazlo **después** de confirmar que tus llaves funcionan, o te puedes quedar fuera).
- Port 2222 — cambiar el puerto por defecto (22) reduce los intentos automáticos de ataque, aunque no reemplaza otras medidas de seguridad.
- Después de cualquier cambio, se reinicia el servicio: `sudo systemctl restart ssh`.

Advertencia importante: siempre prueba una conexión nueva en una segunda terminal antes de cerrar tu sesión actual, para no quedarte bloqueado fuera del servidor si algo salió mal en la configuración.

Ejemplos

```
# Editar el archivo de configuración (con permisos de administrador)
sudo nano /etc/ssh/sshd_config

# Dentro del archivo, buscar y modificar estas líneas:
# PermitRootLogin no
# PasswordAuthentication no
# Port 2222

# Guardar (Ctrl+O, Enter) y salir (Ctrl+X) en nano

# Verificar que la configuración no tiene errores de sintaxis
sudo sshd -t
```

```
# Reiniciar el servicio para aplicar cambios
sudo systemctl restart ssh

# Si cambiaste el puerto, conéctate especificándolo
ssh -p 2222 matias@192.168.1.50
```

Tarea / Proyecto: “Servidor SSH endurecido”

En tu servidor de práctica (con tus llaves ya configuradas y funcionando):

1. Desactiva el login root directo.
 2. Desactiva la autenticación por contraseña (solo después de confirmar que tu llave funciona).
 3. Cambia el puerto por defecto.
 4. Documenta en `conexion_etapa3.md` cada cambio, por qué lo hiciste, y cómo verificaste que no te quedaste bloqueado antes de cerrar tu sesión original.
-

Etapas 4: Túneles SSH y reenvío de puertos

Teoría

SSH no solo sirve para abrir una terminal remota — también puede “envolver” tráfico de red dentro de una conexión cifrada, algo llamado **túnel** o *port forwarding*.

- **Reenvío local** (-L) — accedes a un servicio que corre en el servidor remoto, como si estuviera en tu propia máquina. Útil para acceder a una base de datos remota de forma segura sin exponerla a internet.
- **Reenvío remoto** (-R) — al revés: expones un servicio de tu máquina local hacia el servidor remoto.

Piensa en un túnel SSH como un pasillo secreto y seguro entre dos edificios: en vez de que el tráfico viaje “a la vista” por internet, viaja escondido dentro de la conexión SSH ya cifrada.

Ejemplos

```
# Reenvío local: acceder a un servicio que corre en el puerto 5432 (PostgreSQL)
# del servidor remoto, como si estuviera en tu puerto local 5433
ssh -L 5433:localhost:5432 matias@192.168.1.50

# Ahora, en otra terminal de tu máquina local, puedes conectarte como si
# la base de datos estuviera en tu propia computadora:
# psql -h localhost -p 5433

# Reenvío remoto: exponer un servicio local (puerto 8080) hacia el servidor remoto
ssh -R 9090:localhost:8080 matias@192.168.1.50
```

Tarea / Proyecto: “Acceso seguro a un servicio remoto”

En tu servidor de práctica, instala algo simple que escuche en un puerto (por ejemplo, el nginx de la guía de Linux, en el puerto 80). Desde tu máquina cliente:

1. Crea un túnel local que te permita acceder a ese servicio a través de un puerto distinto en tu máquina.
 2. Confirma con curl o el navegador que puedes ver el contenido a través del túnel.
 3. Documenta en `conexion_etapa4.md` el comando exacto y explica, con tus palabras, por qué esto es más seguro que exponer directamente el puerto 80 a internet.
-

Etapa 5: Transferencia de archivos (SCP y SFTP)

Teoría

Además de abrir una terminal remota, SSH permite mover archivos de forma segura entre dos máquinas:

- `scp` (*secure copy*) — copia archivos entre tu máquina y un servidor remoto, con una sintaxis parecida a `cp`.
- `sftp` — abre una sesión interactiva de transferencia de archivos, parecida a un cliente FTP tradicional pero cifrada.
- `rsync` — herramienta más avanzada para sincronizar carpetas completas, copiando solo lo que cambió (más eficiente para respaldos grandes).

Ejemplos

```
# Copiar un archivo de tu máquina local al servidor remoto
scp archivo.txt matias@192.168.1.50:/home/matias/

# Copiar un archivo del servidor remoto a tu máquina local
scp matias@192.168.1.50:/home/matias/reporte.txt ./

# Copiar una carpeta completa (con -r)
scp -r carpeta_proyecto matias@192.168.1.50:/home/matias/

# Sesión interactiva SFTP
sftp matias@192.168.1.50
# Dentro de sftp: put archivo.txt (subir), get archivo.txt (bajar), ls, exit

# Sincronizar una carpeta con rsync (más eficiente para carpetas grandes)
rsync -avz carpeta_proyecto/ matias@192.168.1.50:/home/matias/carpeta_proyecto/
```

Tarea / Proyecto: “Sincronización de un proyecto”

1. Copia un archivo individual a tu servidor de práctica con `scp`.
 2. Copia una carpeta completa con `scp -r`.
 3. Usa `rsync` para sincronizar esa misma carpeta después de modificar solo un archivo dentro (observa que `rsync` es más rápido porque no vuelve a copiar todo).
 4. Documenta las diferencias que notaste entre `scp` y `rsync` en `conexion_etapa5.md`.
-

Etapa 6: Configuración del cliente (~/.ssh/config)

Teoría

Cuando administras varios servidores, escribir la IP completa y opciones cada vez es tedioso. El archivo ~/.ssh/config en tu máquina cliente te permite crear “alias” con toda la configuración de conexión guardada.

Ejemplos

```
# Editar (o crear) el archivo de configuración del cliente
nano ~/.ssh/config
```

Contenido de ejemplo dentro del archivo:

```
Host servidor-practica
    HostName 192.168.1.50
    User matias
    Port 2222
    IdentityFile ~/.ssh/id_ed25519

Host servidor-produccion
    HostName 203.0.113.10
    User devops
    Port 22
    IdentityFile ~/.ssh/id_ed25519_produccion
```

Con esta configuración, en vez de escribir el comando completo, ahora basta con:

```
ssh servidor-practica
```

Tarea / Proyecto: “Cliente SSH organizado”

Configura tu archivo ~/.ssh/config con al menos dos entradas (puedes usar servidores de práctica distintos, o el mismo con distinta configuración). Documenta en conexion_etapa6.md cómo quedó tu archivo (sin exponer IPs reales sensibles si no quieres) y confirma que puedes conectarte usando solo el alias.

Proyecto final integrador: “Servidor de práctica asegurado y documentado”

Sobre el mismo servidor de la guía de Linux (o uno nuevo), documenta en un README.md extenso, como bitácora completa:

1. Acceso configurado solo por llave SSH, sin contraseña.
2. Root login deshabilitado, puerto cambiado del 22 por defecto.
3. fail2ban instalado y configurado para bloquear IPs con intentos fallidos repetidos.
4. Un túnel SSH funcional documentado para acceder a un servicio interno.
5. Tu archivo ~/.ssh/config con al menos ese servidor configurado como alias.

Este proyecto, junto con el de la guía de Linux, forma la base de tu portafolio DevOps: un servidor Debian configurado, endurecido y accesible de forma segura — exactamente el tipo de evidencia que un entrevistador de DevOps junior quiere ver.

Recursos recomendados

- **Documentación oficial de OpenSSH:** man.openbsd.org/ssh
- **fail2ban:** instala con `sudo apt install fail2ban` y revisa su configuración en `/etc/fail2ban/jail.local`.
- **Siguiente paso natural:** con Linux y SSH dominados, sigue con **Docker** — contenedores que se administran, en buena parte, sobre servidores a los que llegarás por SSH.

Cómo subir cada proyecto a GitHub

```
cd carpeta_del_proyecto
git init
git add .
git commit -m "Agrego documentación de la etapa X"
git branch -M main
git remote add origin https://github.com/MatiasGonzalez-Dev/ssh-devops-practica.git
git push -u origin main
```